॥ ॐ नमो भगवते वासुदेवाय नमः ॥

NLPU — 60M

Q1)(b) Zipfs Law

Ans 1)(b) Zipfs Law describes how words are distributed in natural languages.

Zipfs Law states that :

"The frequency of a word is inversely proportional to its rank in the frequency table."

Mathematically : $f(r) \propto \frac{1}{r}$

- $f(r)$ = frequency of the word
- r = rank of the word in the frequency list.

→ This means the most frequent word appears about twice as often as the second most frequent word.

1) It helps understand word frequency distribution by explaining • statistical structure of natural languages.

2) It helps in efficient vocabulary handling because most words are rare as NLP systems focus on frequent words for modeling & rare words are handled by smoothing.•

3) Zipfs distribution guides n-gram language models & probability estimation of words which improves tasks like speech recognition, machine translation & text generation.

Q2)(a) Heap's Law —— 3M

Ans 2)(a) Heaps Law describes how the size of vocabulary grows as the size of the text corpus grows.

Heap's Law : $V(n) = K \cdot n^{\beta}$

- $V(n)$ = number of unique words (vocabulary size)
- n = total number of words in the corpus
- K = constant (usually 10 – 100)
- β = exponent (typically 0.4 to 0.6)

This means as more text is processed :

- New words continue to appear
- But the rate of new vocabulary growth slows down.

Heap's Law shows that :

- Vocabulary size grows sub-linearly with corpus size.
- Even very large datasets keep introducing new unique words but increasingly slowly.

Heap's law estimate vocabulary size when processing large corpora.

Q2)(a) What is pre-processing in content of text data & why is it necessary in NLP?

Ans 2)(a) Preprocessing is the process of cleaning & transforming raw text data into a structured & usable format so that ~~machine~~ NLP algorithms can effectively analyze it. It prepares textual data before applying NLP models.

Common preprocessing steps:

- Lowercasing
- Tokenization
- Stop word removal
- Punctuation & digit removal
- Stemming & Lemmatization

* Why Preprocessing is necessary in NLP

- Removes noise & irrelevant information from raw text.
- Reduces vocabulary size, making models more efficient.
- Improves accuracy of NLP models by standardizing text data.
- Converts unstructured text into structured data suitable for NLP algorithms.

→ Preprocessing prepares raw text so NLP systems can process, analyze & learn meaningful patterns from it effectively.

Q2)(b) How is probability theory applied in NLP? — 4M

Ans 2)(b) Probability theory is used in NLP to model uncertainity & predict the likelihood of words, sentences or linguistic patterns in natural language.

- Language Modelling: Probability is used to estimate the likelihood of a sequence of words in a sentence.
- Word Prediction: NLP systems use probabilistic models like n-grams to predict the most probable next word based on previous words.
- Text classification: Algorithms such as Naive Bayes use probability to classify text into categories
- Machine Translation & Speech Recognition: Probability helps determine the most likely translation or interpretation of a sentence from many possible alternatives.

→ Probability theory allows NLP systems to handle ambiguity, estimate word likelihoods & make predictions, enabling tasks such as language modelling, classification & translation.

Q2)(b) Stemming & Lemmatization in NLP —— 4M

Ans2)(b)

Stemming	Lemmatization
• Reduces a word to its root form by cutting off prefixes or suffixes.	Reduces a word to its base or dictionary form (lemma) using linguistic rules.
• Uses simple rule-based truncation	Uses vocabulary & morphological analysis.
• Less accurate	More accurate
• May produce non meaningful ~~words~~ words.	Produces valid dictionary words.
• Faster & simpler	Slower but more linguistically correct.
• Quickly removes word endings to obtain a root form.	Uses linguistic analysis to return the correct base word (lemma).

Q2) (c) Various categoris of MT? Parallel corpus? Statistical MT. — 8M

Ans2) (c) Categories of Machine Translation:

1) Rule Based Machine Translation (RBMT):

- Uses linguistic rules, grammar & bilingual dictionaries.
- Translation is performed ~~during~~ using manually written rules.
- Example: SYSTRAN systems.

2) Statistical Machine Translation (SMT):

- Uses statistical models learned from bilingual text corpora.
- Probabilities determine the most likely translation.

3) Example Based Machine Translation (EBMT):

- Translates by comparing input sentences with previously translated examples.
- Uses analogy & example matching.

4) Neural Machine Translation

- Uses deep learning models such as RNNs, LSTMs or transformers
- Learns translation directly from large datasets & produces fluent translations.

A parallel corpus is a dataset containing sentences in one language aligned with their translation in another language

- Parallel corpora provide paired examples of source & target language sentences used to train MT models.
- They help determine which words or phrases correspond between two languages.
- In statistical models, they help estimate translate probabilities
- More parallel data allows the system to learn better translation patterns.

* Statistical Machine Translation

It automatically translates text from one language to another using statistical models trained on bilingual corpora.

Given a sentence in the source language, the system finds the most probable translation in the target language.

The goal is to find the sentence H that maximizes the probability given source sentence S:

$$\text{argmax}_H \; P(H|S)$$

Using Bayes Theorem

$$P(H|S) = \frac{P(S|H) \cdot P(H)}{P(S)}$$

- $P(S|H)$ = Translation Model (probability that target sentence translates to source sentence).
- $P(H)$ = Language Model (probability of target language sentence)
- $P(S)$ = Constant for a given input sentence.

Q3)(b) Provide examples of probabilistic language models used in various NLP tasks. ——— 4M

Ans3)(b) Probabilistic language models estimate the probability of word sequences in a language. They are widely used in many NLP tasks

1) Unigram Language Model

- Considers each word independently
- Example probability: $P(w_1, w_2, w_3) = P(w_1) \times P(w_2) \times P(w_3)$
- Applications are basic text generation & word frequency analysis.

2) Bigram Language Model

- Calculates probability of a word based on one previous word.

$$P(w_n \mid w_{n-1})$$

Example: "I want to eat" → Probability of eat depends on to.

- Applications are speech recognition & predictive typing.

3) Trigram Language Model

- Uses previous two words to predict the next word

$$P(w_n \mid w_{n-1}, w_{n-2})$$

- Applications are Machine translation & text prediction.

4) Hidden Markov Model (HMM)

- Probabilistic sequence model used to predict hidden states.

Q3)(C) In text classification, a confusion matrix summarizes the prediction results of a classification model. It contain four values ——— 8M

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

- TP: correctly predicted positive instances
- TN: correctly predicted negative instances
- FP: incorrectly predicted positive instances
- FN: incorrectly predicted negative instances

1) Accuracy measures the overall correctness of the model

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

2) Precision measures how many predicted positives are actually correct. High precision means few false positives.

$$\text{Precision} = \frac{TP}{TP + FP}$$

3) Recall measures how many actual positives were correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN}$$

High recall means few false negatives.

4) F1-Score

F1-score is the harmonic mean of precision & recall, balancing both metrics

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

→ These metrics are commonly used to evaluate text classification models such as Spam detection, sentiment analysis & document categorization.